



Python Project 3 - Using Monte Carlo to Simulate Random Processes

This project has two parts. You may choose to do either. If you do both, then extra credit will be awarded. Also you have the option to work with another person with this project. In that case, only one code need be submitted.

The first involves using MC to simulate a game of chance where we "draw" a ball out of a bin of 100 balls with each ball numbered from 1 to 100. One goal is to determine the average amount of money the player has remaining after playing a fixed number of times. In addition, we want to determine the discrete expected value when we simulate playing and show that it is close to the theoretical expected value.

The second involves using MC to estimate the amount of commission that a company expects to have to pay. Here the sales rep's commission is based upon how close their actual sales are to their target sales. We first want to simulate the sales for each rep and compute the total amount of commission. Then we want to do this many times and average the total commissions from all the simulations.

Part I - Using Monte Carlo to simulate a game of chance

Suppose we have a game where there are 100 balls with numbers 1 through 100. The rules of the game are simple. If you draw a ball with a number 52, 53, ... , 100 then you win; otherwise the house wins. We can easily compute the probability of winning:

- Player has 49 out of 100 chances of winning so probability is 0.49
- House has 51 out of 100 chances of winning so probability is 0.51

Clearly the house has an edge which we can compute using the expected value. If you bet \$100 then the expected value is

| expected value = $(100)(0.49) - (100)(0.51) = -0.02(100) = -2$

So with this bet, you should expect to lose \$2 per game.

The goal is to approximate your ending funds if you decide you are only going to play a certain number of games. For example, suppose you are only going to play 5 times then we could simulate the game for 5 times. However, this would only give us the remaining funds for one example of playing 5 games. Instead we want to do this many times and average the result.

Assume that the starting funds are 10,000 dollars and we always wager 100 dollars per game.

We solve this problem in the following steps:

- Step 0. Import library and set seed
- Step 1. Define a function which returns True if the player wins and False otherwise.
- Step 2. Define a function which simulates playing the game a given number of times; it uses the function in Step 1 and adjusts the funds available depending if the player wins or loses. Function returns the amount of funds remaining after playing the given number of times and the number of times the player wins/loses.
- Step 3. Write code to call the function in Step 2 for 5 games. Output the funds remaining after 5 games and number of wins/loses.
- Step 4. Modify the code in Step 4 to do this simulation (of playing 5 games) 100 times. Output the average of the funds remaining after 5 games and the probability of winning & losing; output the expected value for this simulation.

The results you get in Step 4 do NOT match our theoretical expected value of losing 2 dollars per game. You are asked to make modification(s) to your code in Step 4 so that your discrete expected value is close to the theoretical value.

```
# import random library and set seed
import random
```

```

# Step 1.
# Define a function which randomly draws a ball with an integer between 1 and 100 on it;
# then returns True if Player
# wins and returns False if House wins.
# Add a print statement to function to print random integer and test out function
# Add comment statements to function
#
def ball_draw():
    pick=random.randint(1,100)
    if pick >= 52:
        print(pick)
        return True
    else:
        print(pick)
        return False

```

```
ball_draw()
```

76

True

```

# Step 2.
# Create a function which simulates playing the game n_plays times; keep track of number of wins and losses
def simulate_game(n_plays):
    funds=10000
    wins=0
    loses=0
    for i in range(n_plays):
        if ball_draw():
            funds += 100
            wins += 1
        else:
            funds -= 100
            loses += 1
    return funds, wins, loses

```

```
# simulate playing 5 games and print funds remaining, wins, and loses
```

```
funds, wins, loses = simulate_game(5)
print(wins, loses, funds)
```

```

63
95
31
50
72
3 2 10100

```

```
# 1. import random and set seed
import random
random.seed(1234)
```

```
# 2. Input target sales and number of sales persons for upcoming year; low and high
# percentages of sales for generating random numbers
target_sales = [150000, 200000, 95000, 350000, 425000]
n_sales = len(target_sales)
low = 0.85
high = 1.10
```

```
# 3. Generate a random sales amount for each rep and check that it is in the correct range
# Use a for loop; round answer to nearest cent

for i in range(n_sales):
    sales = round(random.uniform(low, high) * target_sales[i], 2)
    print(sales)
```

```
163742.01
192036.63
80927.92
377210.4
461047.33
```

```
# 4. Add to your loop in # 3 to determine the commission for each rep  
# (you need an if-elif-else construct to determine rate); keep a running tab of  
# the sum of all commissions  
# round all dollar amounts to nearest cent
```

```
for i in range(n_sales):  
    sales = round(random.uniform(low, high) * target_sales[i], 2)  
    if sales < 0.91 * target_sales[i]:  
        rate = 0.02  
    elif sales < target_sales[i]:  
        rate = 0.03  
    else:  
        rate = 0.04  
    comission = round(sales*rate, 2)  
    print(comission)
```

```
4480.01  
8143.13  
1654.87  
14582.68  
7728.22
```

```
# 5. Add a loop around your code in # 4 to do 10,000 simulations  
# Only print out the average commission for all 10,000 simulations  
# Don't print out all the information from Step # 4 - it will be way too much so remove  
# or comment out those print statements
```

```
n_simulations = 10000  
total_commission = 0  
for i in range(n_simulations):  
    for i in range(n_sales):  
        sales = round(random.uniform(low, high) * target_sales[i], 2)  
        if sales < 0.91 * target_sales[i]:  
            rate = 0.02  
        elif sales < target_sales[i]:  
            rate = 0.03  
        else:  
            rate = 0.04  
        total_commission += round(sales*rate, 2)  
print(total_commission/n_simulations)
```

```
38264.22452999985
```